PROJECT REPORT | CLOUD COMPUTING AND DEVOPS

Enterprise Cloud Infrastructure & DevOps Automation

Docker | Kubernetes | Terraform | GitHub Actions | Prometheus | Grafana

SUBMITTED BY	Sapna Ahamed
PROGRAMME	B.E. Computer Science and Engineering
GRADUATION YEAR	2027
PROJECT DOMAIN	Cloud Computing and DevOps
ENVIRONMENT	GitHub Codespaces / Minikube

github.com/sapnaahamed23-hub/cognevance_enterprise-cloud-infrastructure-devops

Table of Contents

Table of Contents 2

Abstract 4

1. Introduction 5

1.1 Background 5

1.2 Problem Statement 5

1.3 Objectives 5

1.4 Project Scope 6

2. Technologies Used 7

2.1 Docker 7

2.2 Kubernetes 7

2.3 Minikube 7

2.4 Terraform 7

2.5 GitHub Actions 7

2.6 Prometheus 7

2.7 Blackbox Exporter 7

2.8 Grafana 8

2.9 Git and GitHub 8

2.10 YAML and Bash 8

3. System Architecture 9

3.1 Architecture Overview 9

3.2 Application Workflow 9

3.3 Monitoring Workflow 9

3.4 Infrastructure Configuration Workflow 10

4. Implementation 11

4.1 Development Environment 11

4.2 Docker Containerization 11

4.3 Kubernetes Deployment 11

4.4 Terraform Configuration 12

4.5 Continuous Integration 12

4.6 Monitoring Configuration 12

4.7 Persistent Storage 12

4.8 Backup and Recovery . . . 13
5. Testing and Validation . . . 14
5.1 Purpose . . . 14
5.2 Kubernetes Node Validation . . . 14
5.3 Application Replica Validation . . . 14
5.4 HTTP Endpoint Validation . . . 14
5.5 Blackbox Exporter Validation . . . 14
5.6 Prometheus Target Validation . . . 14
5.7 Grafana Validation . . . 15
5.8 Terraform Validation . . . 15
5.9 Recovery Validation . . . 16
5.10 GitHub Actions Validation . . . 16
5.11 Summary of Validation . . . 16
6. Prometheus Query . . . 17
7. Results and Discussion . . . 18
8. Limitations . . . 19
9. Future Enhancements . . . 20
10. Conclusion . . . 21
References . . . 22
Project Repository . . . 22

Abstract

This project, titled *Enterprise Cloud Infrastructure & DevOps Automation*, demonstrates the practical implementation of cloud-native application deployment, containerization, Kubernetes orchestration, Infrastructure as Code (IaC), continuous integration, and infrastructure monitoring.

The application is packaged using Docker and deployed to a Kubernetes cluster running through Minikube in GitHub Codespaces. The Kubernetes deployment uses two application replicas, a ClusterIP Service, health probes, resource requests and limits, and container security controls.

Monitoring is implemented using Prometheus, Blackbox Exporter, and Grafana. Blackbox Exporter probes the application's HTTP endpoint, Prometheus collects the resulting metrics, and Grafana presents the monitoring information through a dashboard. Terraform is used for Kubernetes configuration management and validation, while GitHub Actions automates the repository's CI checks.

The project follows a free-first approach to minimize infrastructure costs and provide practical experience without requiring a paid public cloud environment. Backup manifests and a recovery script are also included, with basic validation performed on the recovery materials.

The project demonstrates foundational Cloud and DevOps practices in an educational environment. It does not claim production-scale availability, production cloud provisioning, or a completed end-to-end disaster-recovery exercise.

Keywords: Cloud Computing, DevOps, Docker, Kubernetes, Minikube, Terraform, GitHub Actions, Prometheus, Blackbox Exporter, Grafana, Infrastructure as Code.

1. Introduction

1.1 Background

Modern software delivery involves more than developing an application. Applications must be packaged, deployed consistently, monitored, and maintained throughout their lifecycle. Cloud Computing and DevOps practices help teams automate these activities, improve repeatability, and identify operational issues.

Containerization provides a consistent runtime environment, while Kubernetes manages containerized workloads. Infrastructure as Code makes infrastructure configuration easier to version and validate. Continuous Integration automates checks whenever code changes are introduced. Monitoring tools provide visibility into application availability and operational metrics.

This project combines these concepts into a hands-on cloud-native lab using free-first tools.

1.2 Problem Statement

Manually configuring application environments, deploying containers, checking service availability, and maintaining infrastructure can become repetitive and error-prone. Without automated validation and monitoring, configuration problems and application availability issues may go unnoticed.

The project addresses these learning objectives by implementing a containerized application, Kubernetes deployment, automated CI validation, infrastructure configuration, monitoring, and documented recovery procedures in a reproducible development environment.

1.3 Objectives

The primary objectives are:

1. Containerize a web application using Docker.
2. Deploy the application to Kubernetes using Minikube.
3. Configure two application replicas and a ClusterIP Service.
4. Implement Kubernetes readiness and liveness probes.
5. Apply resource requests, resource limits, and baseline container security controls.
6. Use Terraform for Kubernetes configuration management and validation.
7. Configure GitHub Actions for automated CI checks.
8. Monitor HTTP endpoint availability using Blackbox Exporter and Prometheus.
9. Visualize monitoring metrics using Grafana.
10. Document Kubernetes backup and recovery procedures.
11. Demonstrate Cloud and DevOps practices without depending on paid cloud resources.

1.4 Project Scope

The project covers containerization, Kubernetes workload orchestration, infrastructure configuration validation, continuous integration, service availability monitoring, dashboard visualization, and recovery documentation.

The application runs in a Minikube environment hosted through GitHub Codespaces. It is an educational implementation rather than a production deployment on AWS, Microsoft Azure, or Google Cloud.

2. Technologies Used

2.1 Docker

Docker is used to package the application and its runtime environment into a container image. The Dockerfile uses a Python 3.12 slim base image and configures the application to run as a non-root user. A container health check is also defined.

2.2 Kubernetes

Kubernetes manages the application workload. A Deployment maintains two application replicas, while a ClusterIP Service provides internal access to the application. Health probes help Kubernetes determine whether the application is ready to receive traffic and whether it should be restarted. The deployment also specifies resource requests and limits and applies baseline pod security settings.

2.3 Minikube

Minikube provides the Kubernetes cluster used for this project. It enables Kubernetes deployment and monitoring workflows without requiring a managed production cluster.

2.4 Terraform

Terraform is used for Infrastructure as Code with the Kubernetes provider. The project includes Terraform configuration files and uses initialization, validation, and planning commands to check the configuration. The project documentation reports that the Terraform configuration was applied to create a Kubernetes ConfigMap and that a subsequent plan reported no changes at the time of validation.

2.5 GitHub Actions

GitHub Actions provides automated CI checks through a repository workflow. The project documentation records a successful workflow run involving Terraform validation and Docker image building. The workflow does not publish the image to a container registry or automatically deploy the application to a Kubernetes cluster.

2.6 Prometheus

Prometheus collects and stores monitoring metrics. In this project, it scrapes metrics exposed by Blackbox Exporter and provides a data source for Grafana.

2.7 Blackbox Exporter

Blackbox Exporter performs an HTTP probe against the application endpoint. It reports whether the probe succeeds and provides related information, including the HTTP response status code.

2.8 Grafana

Grafana visualizes monitoring metrics retrieved from Prometheus. The project includes a dashboard titled **Enterprise Cloud Infrastructure Monitoring**. The dashboard includes panels for:

- Probe success.
- Monitored target status.
- HTTP response status code.

A PersistentVolumeClaim is configured for Grafana's data directory to provide persistent storage across ordinary pod replacements, provided the underlying storage remains available.

2.9 Git and GitHub

Git and GitHub are used for source control, repository hosting, change tracking, workflow execution, and project documentation.

2.10 YAML and Bash

YAML is used to define Kubernetes resources and configuration. Bash is used to invoke the recovery script and perform command-line operations.

3. System Architecture

3.1 Architecture Overview

The system consists of a source repository, CI workflow, container image, Kubernetes cluster, application workload, monitoring components, and dashboard. The architecture follows the logical flow shown in Figure 1.

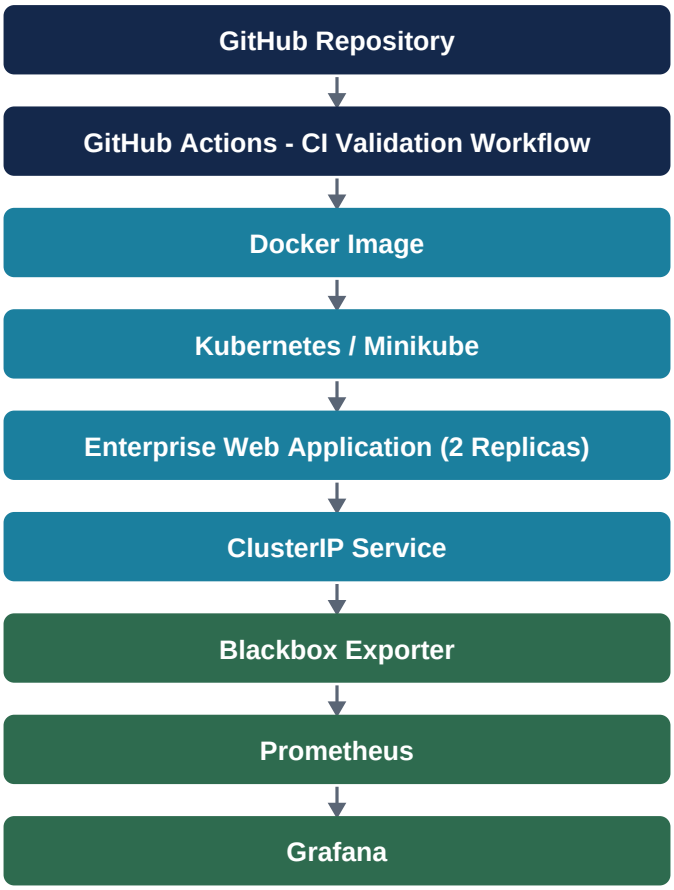


Figure 1: Logical architecture of the project

The architecture illustrates the major components and their logical relationships. It does not represent an automatically deployed production cloud environment.

3.2 Application Workflow

The application is packaged as a Docker image and deployed through Kubernetes manifests. The Deployment maintains two replicas of the application, and the ClusterIP Service provides an internal access point. Kubernetes readiness and liveness probes monitor the application endpoint. Resource requests and limits define resource expectations for the workload.

3.3 Monitoring Workflow

The monitoring process follows these steps:

1. Blackbox Exporter sends an HTTP probe to the configured application endpoint.
2. Blackbox Exporter exposes probe metrics.

3. Prometheus scrapes the exporter metrics.
4. Grafana queries Prometheus.
5. The Grafana dashboard displays the observed monitoring values.

The monitoring setup provides visibility into endpoint availability and response status at the time of observation.

3.4 Infrastructure Configuration Workflow

Terraform configuration files describe Kubernetes resources managed through the Kubernetes provider. Terraform initialization prepares the working directory, validation checks configuration syntax and consistency, and planning identifies proposed infrastructure changes.

GitHub Actions provides automated CI validation. The workflow does not automatically provision a production cloud environment or deploy the application to an external cluster.

4. Implementation

4.1 Development Environment

The project was developed using GitHub Codespaces and Minikube. This approach allowed Kubernetes and container workflows to be explored without requiring the development laptop to run the entire environment locally. The implementation uses free-first tools and does not depend on paid public cloud infrastructure.

4.2 Docker Containerization

The Dockerfile packages the application using Python 3.12 slim. The application is served from the application directory using Python's built-in HTTP server on port 8000. The container configuration includes:

- A lightweight base image.
- Non-root application execution.
- A health check against the application endpoint.
- A defined application port.

These choices demonstrate basic containerization and security practices.

4.3 Kubernetes Deployment

The Kubernetes configuration defines a namespace named `enterprise-dev` and a Deployment named `enterprise-cloud-app`. The Deployment specifies two replicas of the application. A ClusterIP Service named `enterprise-cloud-service` provides internal access to the application. The deployment configuration includes:

- Two application replicas.
- Container port 8000.
- CPU and memory resource requests and limits.
- Liveness and readiness probes.
- Non-root execution.
- Disabled privilege escalation.
- Dropped Linux capabilities.
- RuntimeDefault seccomp profile.

The probes use HTTP requests to check the application endpoint. Readiness probes help determine when a pod can receive traffic, while liveness probes help Kubernetes detect an application that may need restarting.

These configurations demonstrate useful Kubernetes practices, but they do not by themselves establish production readiness.

4.4 Terraform Configuration

The project contains Terraform configuration files under the `terraform/` directory. Terraform was used to manage Kubernetes configuration, including a ConfigMap as documented in the implementation notes. The configuration was initialized and validated, and a subsequent plan was reported to contain no changes at the time of testing. The main commands used during validation were:

```
terraform -chdir=terraform init
terraform -chdir=terraform validate
terraform -chdir=terraform plan
```

The Terraform Kubernetes provider requires access to a compatible Kubernetes cluster and a valid kubeconfig when interacting with the cluster. Terraform validation and planning are useful checks, but they do not demonstrate that production cloud resources have been provisioned.

4.5 Continuous Integration

The project uses a GitHub Actions workflow stored at:

```
.github/workflows/ci.yml
```

The workflow automates CI checks described in the repository documentation, including Terraform validation and Docker image building. A successful workflow run was recorded during implementation. The current workflow does not publish the image to a registry or automatically deploy the application to Kubernetes.

4.6 Monitoring Configuration

The monitoring stack consists of Prometheus, Blackbox Exporter, and Grafana. Blackbox Exporter probes the configured application endpoint. Prometheus collects the exporter metrics, and Grafana uses Prometheus as its data source. The Grafana dashboard displays the following metrics:

- `probe_success`
- `up`
- `probe_http_status_code`

The monitoring components run within the Kubernetes learning environment. The monitoring results indicate observed status at a particular point in time and should not be interpreted as evidence of long-term uptime or a formal service-level agreement.

4.7 Persistent Storage

A PersistentVolumeClaim named `grafana-storage` is configured in the monitoring namespace. The claim requests 1 GiB of persistent storage with ReadWriteOnce access and uses the configured `standard` storage class.

Grafana mounts the volume at `/var/lib/grafana`. This configuration is intended to preserve Grafana data across ordinary pod replacements while the PersistentVolumeClaim and underlying

storage remain available. Persistent storage alone does not guarantee protection against storage loss or provide a complete disaster-recovery solution.

4.8 Backup and Recovery

The repository includes Kubernetes backup manifests and a recovery script:

```
scripts/restore-kubernetes-backup.sh
```

The recovery materials were checked through manifest dry-run validation and shell syntax validation. The recovery script applies Kubernetes resources. Before executing it, the operator should inspect the backup files, confirm the active Kubernetes context, and verify the target cluster.

> **Note:** A complete end-to-end recovery drill has not been performed. The project documents a recovery procedure but does not claim that full disaster recovery has been demonstrated.

5. Testing and Validation

5.1 Purpose

Testing was performed to check whether the application, Kubernetes deployment, monitoring stack, Terraform configuration, CI workflow, and recovery materials behaved as expected within the project environment. The following results summarize the checks reported during implementation.

5.2 Kubernetes Node Validation

The Minikube node was verified to have reached the `Ready` state. **Significance:** the Kubernetes node was available to schedule workloads at the time of observation.

5.3 Application Replica Validation

The application Deployment was observed with two replicas running and ready. **Significance:** Kubernetes had achieved the configured application replica count at the time of observation.

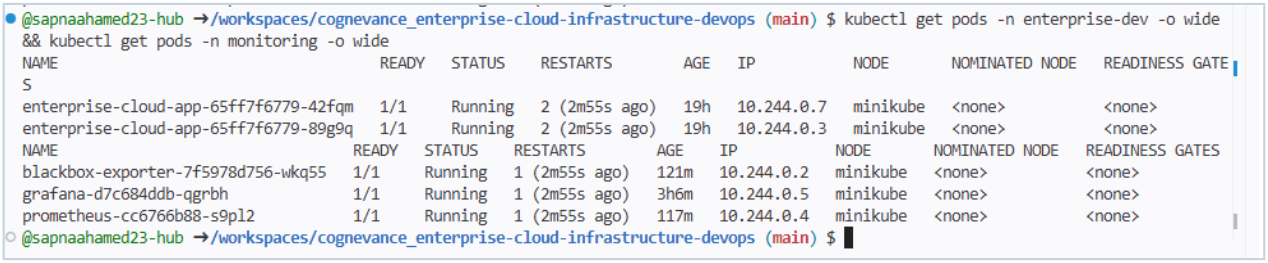


Figure 2: *Kubernetes pods in the enterprise-dev and monitoring namespaces, all Running (1/1 ready)*

5.4 HTTP Endpoint Validation

The application endpoint returned HTTP status code `200`. **Significance:** the application endpoint returned a successful HTTP response at the time of testing.

5.5 Blackbox Exporter Validation

The Blackbox Exporter probe reported:

```
probe_success = 1
```

Significance: the configured HTTP availability probe succeeded at the time of observation.

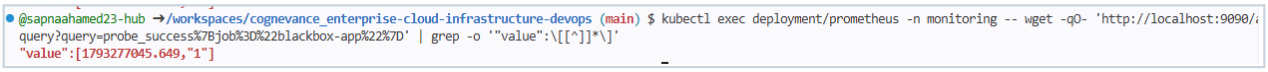


Figure 3: *Prometheus query probe_success{job="blackbox-app"} returning value 1 (terminal output)*

5.6 Prometheus Target Validation

The configured monitoring targets were observed reporting `UP`. **Significance:** Prometheus and the configured monitoring targets were reachable at the time of observation.

5.7 Grafana Validation

The Grafana dashboard was created, the Prometheus data source test succeeded, and the monitoring metrics were displayed. The dashboard included the probe success value, target status values, and HTTP response status code. The following values were observed:

Metric	Observed value	Interpretation
`probe_success`	`1`	The configured HTTP probe succeeded at that time.
`up`	`1, 1`	Two monitored targets were reporting as reachable at that time.
`probe_http_status_code`	`200`	The application endpoint returned a successful HTTP response.

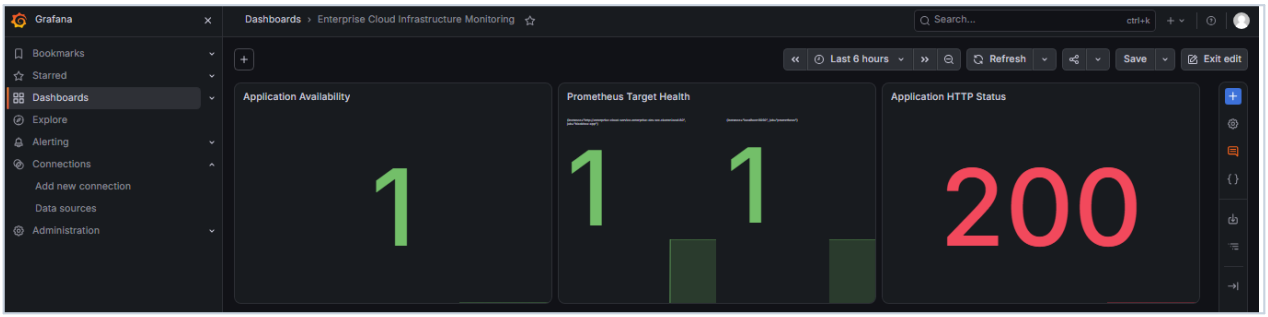


Figure 4: Grafana dashboard: Enterprise Cloud Infrastructure Monitoring (Application Availability, Prometheus Target Health, Application HTTP Status)

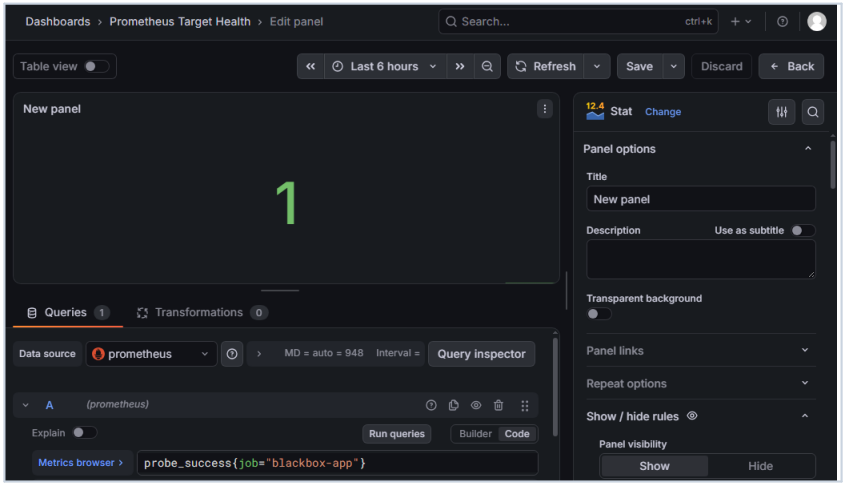


Figure 5: Grafana panel editor with the Prometheus query probe_success{job="blackbox-app"}

These values are point-in-time observations and do not prove continuous availability.

5.8 Terraform Validation

Terraform initialization and validation were completed, and a subsequent plan was reported to show no changes. **Significance:** the configuration passed validation and was aligned with the observed Terraform-managed state at the time of planning.

5.9 Recovery Validation

The Kubernetes backup and recovery manifests were checked using dry-run validation, and the recovery script passed shell syntax validation. **Limitation:** a complete recovery drill was not performed. These checks validate basic aspects of the recovery materials but do not prove that a full recovery will succeed under failure conditions.

5.10 GitHub Actions Validation

A GitHub Actions workflow run was reported to have completed successfully for a recorded commit. **Significance:** the configured workflow passed for that run. This result does not establish that the project has an automated production deployment pipeline.

5.11 Summary of Validation

Component	Validation result
Minikube	Node reached Ready
Kubernetes application	Two replicas running and ready
Application endpoint	HTTP 200
Blackbox Exporter	probe_success = 1
Prometheus	Configured targets reported UP
Grafana	Dashboard created and data source test succeeded
Terraform	Validation passed; subsequent plan reported no changes
Recovery materials	Manifest dry-run and script syntax checks completed
GitHub Actions	A workflow run completed successfully

The results indicate that the main components operated successfully during the reported checks. They do not constitute production performance testing, a security audit, or a completed disaster-recovery exercise.

6. Prometheus Query

The following Prometheus query was used to inspect the application's HTTP probe result:

```
probe_success{job="blackbox-app"}
```

A value of `1` indicates that the configured probe succeeded at the time the metric was collected. A value of `0` indicates that the probe did not succeed. The exact result depends on the monitoring configuration and the state of the target at the time of observation.

7. Results and Discussion

The project brought together several core Cloud and DevOps technologies within one learning environment.

The Docker container provided a consistent application runtime. Kubernetes managed the application replicas and provided internal service access. Health probes and resource configuration demonstrated basic workload management practices.

Terraform provided an Infrastructure as Code workflow for Kubernetes configuration, while GitHub Actions automated CI validation. The Prometheus and Blackbox Exporter integration made endpoint availability observable, and Grafana presented the monitoring data through a dashboard.

The recovery manifests and script provided a documented starting point for Kubernetes recovery operations. Basic validation was performed, although a complete recovery drill remains future work.

Overall, the project demonstrates practical familiarity with the tools and workflows involved in containerization, orchestration, infrastructure configuration, CI validation, monitoring, and operational documentation.

8. Limitations

The following limitations define the current scope of the project:

1. **Local Kubernetes environment:** the application runs on Minikube in GitHub Codespaces rather than on production cloud infrastructure.
2. **No production cloud provisioning:** the project does not claim to provision production resources on AWS, Azure, or Google Cloud.
3. **No registry publishing:** the CI workflow does not publish versioned container images to a container registry.
4. **No automatic deployment:** the CI workflow does not automatically deploy the application to a Kubernetes cluster.
5. **Recovery drill incomplete:** a complete end-to-end disaster-recovery exercise has not been performed.
6. **No production security audit:** the project has not undergone a comprehensive security assessment.
7. **No production-scale availability testing:** the monitoring observations do not establish long-term uptime or production service-level objectives.
8. **Educational scope:** the configuration demonstrates foundational practices and may require additional design, testing, security, and operational controls before production use.

These limitations are stated to distinguish the demonstrated implementation from planned or production-grade capabilities.

9. Future Enhancements

The following improvements could extend the project in future iterations:

1. Implement automated deployment to a suitable Kubernetes environment.
2. Publish versioned Docker images to a container registry.
3. Explore deployment to a public cloud using suitable free-tier resources.
4. Add Prometheus alerting rules and notification channels.
5. Expand automated application tests.
6. Perform and document an end-to-end Kubernetes recovery drill.
7. Improve secrets management and access-control practices.
8. Add structured application logging and further monitoring.
9. Document resource utilization and performance under controlled test conditions.
10. Develop a repeatable setup guide for reproducing the environment.

These are proposed improvements and are not represented as completed features.

10. Conclusion

The *Enterprise Cloud Infrastructure & DevOps Automation* project demonstrates a practical combination of Cloud Computing and DevOps concepts using Docker, Kubernetes, Minikube, Terraform, GitHub Actions, Prometheus, Blackbox Exporter, and Grafana.

The application was containerized and deployed with two Kubernetes replicas. Health probes, resource configuration, and basic container security controls were implemented. Monitoring provided visibility into HTTP endpoint availability, while Terraform and GitHub Actions supported infrastructure configuration validation and CI checks.

The project also includes backup and recovery documentation, with basic validation performed on the recovery materials. A complete recovery drill remains an identified area for improvement.

The implementation provides a hands-on foundation for understanding cloud-native workflows in a free-first environment. It is best described as an educational Kubernetes and DevOps lab rather than a production cloud platform.

The project demonstrates the ability to work with multiple DevOps tools, document implementation decisions, interpret monitoring results, and identify limitations and future improvements.

References

1. Docker Documentation — https://docs.docker.com/
2. Kubernetes Documentation — https://kubernetes.io/docs/
3. Minikube Documentation — https://minikube.sigs.k8s.io/docs/
4. Terraform Documentation — https://developer.hashicorp.com/terraform/docs
5. GitHub Actions Documentation — https://docs.github.com/actions
6. Prometheus Documentation — https://prometheus.io/docs/
7. Prometheus Blackbox Exporter — https://github.com/prometheus/blackbox_exporter
8. Grafana Documentation — https://grafana.com/docs/

Project Repository

https://github.com/sapnaahamed23-hub/cognevance_enterprise-cloud-infrastructure-devops

Author: Sapna Ahamed
Programme: B.E. Computer Science and Engineering
Graduation Year: 2027